

Through abstract factory patterns, OOP offers the option to have extended class correctly modify the functionality of the base class and previously instantiated classes.

Polymorphism in System Verilog

Like 'e', System Verilog classes are not automatically constructed and therefore do not use memory before these are explicitly constructed by calling new:

```
TransBaseT obj_TransBaseT = new; //
Handle to instance of Base Class
DbTransT obj_DbReqT = new; //Handle
to instance of Derived Class

obj_TransBaseT.processTrans(); //
Calls the base class method with the
// constraints in base class
obj_DbReqT.processTrans(); // Calls
the derived class method with the
// constraints in derived class
```

Even though one is calling the function from base class, it has a handle of derived class. This is known as 'polymorphism.' OOP users use polymorphism to create factory patterns that mimic AOP functionality of 'e'. In System Verilog, a method called using polymorphism must be declared as virtual.

The sample code for calling the derived class method from base class object is given below:

```
obj_TransBaseT = obj_DbReqT; //
Assigning handle of derived class to
base class
obj_TransBaseT.processTrans(); //
Will call derived class method
```

Controlling factory patterns

Let's say you have a new class 'IdbTransT' derived from 'DbTransT' and you want to instantiate 'IdbTransT' throughout design under the following two conditions:

1. You want changes reflected everywhere in design
2. You want 'DbTransT' to be completely replaced functionality-wise with 'IdbTransT', but you don't want to modify base class 'DbTransT'

In 'e', you can extend the class without changing the name and that changes every instantiation globally. In System Verilog, you can use factory patterns.

The sample code for factory patterns in System Verilog is:

```
class IdbTransT extends DbTransT;
    rand bit [31:0] channelEnables;
    rand bool    isMySpace;
    rand bit [3:0] spcEnable;
    constraint C2 { addr inside
        [32'h000f:32'hf000]}
    constraint C4 { channelEnables
        inside [32'h1:32'hff] }
    function printMe();
        $display("DbTransT : Addr= %h Data
        = %h Wr_Rd=%h ChEn = %h isMySpc = %h
        SpcEn = %h", addr,data,wr_rd,channel
        Enables,isMySpace,spcEnables);
    endfunction
    function processTrans();
        $display("Processing IdbTransT
        Packet")
        printMe();
    endfunction
endclass

class StimGenT #
    (type PACKET = TransBaseT);
    task run;
        PACKET pkt;
        for (int i=0;i<5;i++) begin
            pkt = new;
            assert(pkt.randomize());
            pkt.processTrans();
        end
    endtasks : run
endclass : StimGenT

module top;
    StimGenT #(TransBaseT) obj1 = new;
    stim_gen #(IdbTransT) obj2 = new;

    initial
        begin
            obj1.run;
            obj2.run;
        end
endmodule : top
```

'when' inheritance in OOP

The sample code for 'when' inheritance in 'e' language is:

```
<'
extend DbTransT TransBaseT {
    when SMALL TransBaseT {
        keep data.size() in [0..10];
    }
}
'>
```

The equivalent code in System Verilog would be:

```
class LargeDbTransT extends DbTransT;
    constraint c5 {
        data.size() inside
            [32'h50:32'h100]
    }
    function processTrans();
        printMe();
    endfunction
endclass
```

The following code illustrates that based on the random value of 'Pkt-Size,' you get appropriate extended class handle which is assigned to the base class:

```
typedef enum {SMALL,MEDIUM,LARGE}
PktSize;

class stim fact; //Intermediate
Class
    static function TransBaseT
    create_stim(PktSize ps) ;
        case (ps)
            SMALL : begin
                DbTransT s = new;
                return s;
            end
            MEDIUM : begin
                IdbTransT s = new;
                return s;
            end
            LARGE : begin
                LargeDbTransT s = new;
                return s;
            end
        endcase
    endfunction : create_stim
endclass : stim fact
```

This functionality can be achieved with the post_randomize() function also.

'is also' in OOP

When you want to add more functionality to a function, the 'is also' extension in 'e' is quite handy.